# Constraint-Based Optimization for Weekly Training Program Design
# A Greedy Heuristic Approach to Multi-Objective Scheduling

**Author: Aaron Tobias**

---

## Abstract

Designing effective weekly training programs requires balancing multiple competing factors, including exercise selection, fatigue management, time constraints, recovery requirements, and user-specific goals. In practice, these decisions are often made heuristically rather than through formal optimization methods. This project formulates workout planning as a constrained scheduling problem and proposes a hybrid two-phase approach: a greedy heuristic algorithm to construct feasible baseline plans, followed by a local search refinement to optimize workload distribution. The system integrates hard constraints such as session duration and fatigue limits with soft optimization criteria including movement coverage, exercise priority, and goal alignment. A random feasible baseline is implemented for comparison. Preliminary results demonstrate that the approach produces structured, constraint-compliant schedules while exposing tradeoffs in workload balance and session utilization. This work establishes a foundation for further optimization and analysis in subsequent project phases.

---

## Problem Definition and Motivation

Designing effective training programs is a complex task that requires balancing multiple competing factors, including movement coverage, fatigue management, time constraints, recovery requirements, and individual preferences. In practice, these programs are often constructed heuristically by coaches or practitioners based on experience rather than formal optimization. Drawing from experience in applied training and movement-based environments, it becomes evident that programming decisions are highly interdependent. Each exercise selection influences subsequent decisions due to accumulated fatigue, movement redundancy, and recovery limitations. This creates a structured decision-making problem that closely resembles the Resource-Constrained Project Scheduling Problem (RCPSP) studied in computer science (Pinedo, 2016). Despite this, most real-world training programs are not generated through formal algorithmic frameworks. Instead, they rely on informal heuristics that may not consistently optimize for key outcomes such as balanced workload distribution or efficient use of limited session time. This gap motivates the formulation of training program design as a computational problem.

In this project, weekly workout planning is modeled as a constrained optimization problem in which exercises must be assigned to training sessions under strict feasibility requirements. These include session duration limits, fatigue capacity, equipment availability, and inter-session recovery constraints. Fatigue is modeled as a bounded

resource, reflecting well-established concepts of training load (Impellizzeri et al., 2005; Zatsiorsky & Kraemer, 2006). Additionally, the system must satisfy higher-level objectives such as movement category coverage and alignment with user-specific goals, consistent with research on training frequency and adaptation (Schoenfeld et al., 2016). The system must operate efficiently under bounded computational complexity, producing schedules in near real-time while maintaining correctness under strict feasibility constraints.

---

**Related Work and Industry Context**

The problem of scheduling under constraints has been extensively studied in computer science, particularly in areas such as job scheduling, resource allocation, and combinatorial optimization (Cormen et al., 2009; Garey & Johnson, 1979). Classical approaches include greedy algorithms, dynamic programming, and approximation algorithms for NP-hard problems. Greedy algorithms are commonly used when optimal solutions are computationally expensive to obtain, providing efficient approximations through locally optimal decisions (Kleinberg & Tardos, 2006). Constraint satisfaction frameworks further extend this approach by separating feasibility constraints from optimization objectives (Dechter, 2006).

In industry, similar principles are applied in large-scale systems such as recommendation engines, infrastructure scheduling, and logistics planning (Beyer et al., 2016; Gomez-Uribe & Hunt, 2015). These systems often rely on heuristic methods due to scalability constraints. However, existing approaches typically assume abstract task structures and do not incorporate domain-specific considerations such as fatigue accumulation or recovery constraints. This project addresses that gap by embedding domain-informed constraints into a scheduling framework.

---

**Proposed Solution**

The proposed system constructs weekly training plans using a constraint-based greedy scheduling algorithm, enhanced by a local search optimization. The solution separates decision-making into four layers:

1. 1. A feasibility layer enforcing hard constraints
2. 2. A scoring layer evaluating candidate exercises
3. 3. A construction layer (Greedy heuristic) assembling the initial schedule
4. 4. A refinement layer (Local Search) exploring schedule mutations to escape local minima and balance fatigue (Russell & Norvig, 2020)

This layered approach ensures that all generated plans satisfy strict constraints while prioritizing high-quality selections through heuristic scoring and subsequent refinement.

---

### Algorithms and Data Structures

The core algorithm follows a greedy construction strategy. For each training day, the system iteratively evaluates all feasible exercises and selects the one with the highest heuristic score. The session state is updated after each selection, and the process continues until no additional exercises can be added without violating constraints. The time complexity of this construction phase is O(D x N x K), where D represents the number of training days, N represents the number of exercises, and K represents the number of selections per session. This results in linear scaling with respect to dataset size, making the approach efficient for moderate problem sizes. Following construction, the local search refinement iterates over neighborhood configurations (swaps, insertions) to minimize fatigue variance. Memory usage is similarly efficient, requiring only storage of exercise data, session states, and auxiliary tracking structures such as assigned exercises and category history.

---

### Greedy Design Rationale

The greedy approach is chosen due to the combinatorial complexity of the scheduling problem. Exact optimization methods such as dynamic programming are impractical due to the interaction of multiple constraints. By selecting locally optimal choices, the system efficiently constructs valid baseline schedules. While this does not guarantee global optimality on its own, it produces strong practical results and aligns with established approximation methods in scheduling theory (Kleinberg & Tardos, 2006). The subsequent local search phase bridges the gap between local and global optimality by smoothing out the fatigue load.

---

### Heuristic Scoring Function

The scoring function evaluates candidate exercises based on multiple factors:

- • exercise priority
- • category coverage
- • goal alignment
- • user preferences
- • time fit
- • fatigue penalty

These components are combined into a weighted sum, guiding the greedy selection process. The scoring system reflects domain-informed priorities and approximates long-term value through local decisions.

---

**Constraint Modeling and Domain Representation**

A key aspect of this project is the translation of domain-specific training principles into formal computational constraints. Exercises are modeled as structured entities with attributes influencing feasibility and desirability. Fatigue is treated as a bounded resource within each session, ensuring workload remains within manageable limits (Impellizzeri et al., 2005). Recovery constraints enforce a 48-to-72 hour spacing between similar movement categories to allow for central nervous system and muscle protein synthesis repair (Haff & Triplett, 2015), preventing excessive repetition and reflecting real-world training practices (Zatsiorsky & Kraemer, 2006). Category coverage ensures structural completeness across the week, aligning with research emphasizing the importance of balanced training frequency (Schoenfeld et al., 2016).

---

**System Architecture**

The system is composed of modular components:

Input JSON -> Loader -> Models -> Filters -> Scoring -> Scheduler (Greedy + Refinement) -> Evaluator -> Output

Each module handles a specific responsibility, improving clarity and extensibility.

---

**Experimental Plan**

The experimental evaluation compares the greedy scheduler and the refined local search output to a random feasible baseline. Experiments will be conducted on a standard computing environment across 30-trial batches to ensure statistical consistency and evaluate variance. The target structure focuses on multi-day (e.g., 5-day) training splits. Metrics include:

- • coverage score
- • priority score
- • time utilization
- • fatigue balance (variance minimization)
- • frequency goals (targeting 5 days/week)
- • constraint violations
- • total score

Visualization will include score comparisons and distribution analysis.

---

### Limitations and Future Work

The greedy algorithm is inherently limited by its local decision-making strategy and may produce suboptimal global solutions if unrefined. The fatigue model is simplified as a flat resource metric and does not capture all complex physiological factors. Additionally, the scoring function relies on manually tuned weights, which may not generalize across all use cases. Future iterations of this project will transition from 'Flat Fatigue' optimization to Undulating Periodization models to map against real-world macrocycle planning (Fleck & Kraemer, 2014).

---

### Conclusion

This project demonstrates how training program design can be formulated as a constrained optimization problem. The proposed system provides an efficient and interpretable framework for generating feasible schedules while balancing multiple objectives. The results establish a foundation for further refinement and analysis in subsequent phases.

---

### Implementation Roadmap

Week 1: core structures, loader, constraints

Week 2: scoring, evaluation, baseline

Week 3: refinement, experiments, analysis

---

### Architecture Diagram